Dataset Collision-Checking Workflow

Detection and Attribution of AI-Generated and Human Written Phishing Emails from Legitimate Emails

Mannan Bhardwaj – 25BYB0013
Bhoumik Singh Rajput – 25BYB0120
School of Computer Science Engineering and Information Systems

1 Purpose

Before any Naive Bayes sub-model is trained on `training_set.csv`, the dataset must be checked for *collisions*: cases where the row-level bookkeeping (identifiers, class labels, and the train/holdout split) is internally inconsistent in a way that would silently corrupt training or inflate reported accuracy. This is distinct from the earlier statistical validity checks (Kruskal-Wallis, Chi-Square, the TF-IDF baseline classifier), which ask *whether the features carry real signal*. The workflow below instead asks *whether the dataset's own bookkeeping is self-consistent* — a necessary precondition that has to hold regardless of which features are eventually used. It was run against the corrected (recipient-identity-leak-free) versions of `training_set.csv` and `holdout_set.csv`.

2 The four collision classes

Each email in the dataset is assigned an identifier `id`, computed as a truncated (40-bit) MD5 hash of its body text, prefixed by its source corpus. Four distinct failure modes are checked, in order from most mechanical to most subtle.

1. ID collision (hash collision)

Two rows share the same `id` value but have *different* body text. Since `id` is a hash of the body, this would only happen from a genuine MD5 collision or from a pipeline bug (e.g. the hash computed before a text-normalizing step was applied inconsistently). At the dataset's row count (∼13,000 total), the expected number of genuine hash collisions under a 40-bit hash is approximately

$$\frac{n^2}{2 \cdot 2^{40}} \approx \frac{13000^2}{2 \cdot 2^{40}} \approx 4 \times 10^{-5},$$

i.e. essentially zero. A nonzero count here should be treated as a pipeline bug, not bad luck.

2. Cross-class label collision

The exact same body text appears more than once with *different* class labels (e.g. the identical email tagged both *Legitimate* and *Human-Phishing* in different rows). This is the most serious possible failure: it is a direct contradiction in the ground truth, and any model trained across such rows is being given mutually exclusive answers for the same input.

3. Train/holdout overlap

Any row — checked both by `id` and, independently, by raw body text — appears in *both* `training_set.csv` and `holdout_set.csv`. This would let information used for training leak into the holdout set, inflating any accuracy reported against it.

4. Duplicate content (within a file)

The same body text appears more than once within a single file. Unlike the previous three, this is not automatically a failure — a phishing template genuinely sent to many recipients produces legitimately repeated content — but it is tracked as a regression guard against unintended reintroduction of exact duplicates (the kind removed during corpus cleaning).

5. Near-duplicate template reuse (informational)

Exact-match checks (1–4) miss emails that are identical *except* for a date, dollar amount, or name. These are detected by normalizing each body (lower-casing, collapsing all digits to a single placeholder character, stripping punctuation, collapsing whitespace) and then checking for repeats under that normalized form. This is not treated as a hard failure — template reuse is a realistic property of a phishing corpus — but any near-duplicate cluster that spans *more than one class* is flagged specifically, since that would be equivalent in severity to a cross-class label collision.

3 Results on the current dataset

Table 1 reports the outcome of running this workflow against the corrected `training_set.csv` (4,491 rows) and `holdout_set.csv` (8,465 rows).

Check	**Training set**	**Holdout set**
ID collisions (different content, same id)	0	0
Cross-class label collisions	0	0
Duplicate-content groups (exact match)	0	0
Near-duplicate groups (normalized match)	45 (95 rows)	38 (133 rows)
Near-duplicate clusters crossing class boundary	0	0

Table 1: Collision-check results. The four hard-failure checks (rows 1–3 and the cross-file row below) are all 0.

Cross-file check	**Result**
Train/holdout overlap by `id`	0
Train/holdout overlap by body text	0

All four hard-failure checks pass. The 45 (training) and 38 (holdout) near-duplicate clusters are template-reuse within a single class only — manual inspection confirmed 91 of the 95 affected

training rows are Human-Phishing (consistent with template phishing campaigns sent to multiple recipients) and the remaining 4 are Legitimate; none span class boundaries, so none represent a labeling problem.

4 Re-running the workflow

This check should be re-run any time the dataset-building pipeline (`clean_and_merge.py`) is modified, or before any claim of dataset validity is made in the written paper. It is intentionally kept separate from the statistical-validity checks (Section 3 of the proposal) because it tests a different property: internal bookkeeping consistency, not feature informativeness. A pipeline change that alters row counts, deduplication, or redaction logic should always be followed by re-running both this workflow and the statistical baseline check before new files are treated as final.